The **wait** Construct for Parallel Session Types: Motivation, Design, and Lattice Preservation

Alexandre Zua Caldeira
Independent Researcher
Berlin, Germany
azuacaldeira@gmail.com

March 2026

DRAFT — internal design note

Abstract

We identify two sources of ambiguity in session type grammars that support parallel composition with continuation: (1) the use of **end** inside parallel branches conflates session termination with synchronisation, and (2) syntactic sugar for method sequencing introduces a hidden desugaring rule that interacts poorly with the continuation operator. We resolve both by introducing the **wait** keyword for branch synchronisation and removing the sequencing sugar in favour of explicit branch notation. The resulting *core grammar* has nine constructs, no desugaring rules, and a single-purpose continuation operator that applies exclusively after parallel composition. We prove that continuation preserves the lattice property of the state space via an ordinal-sum argument.

1 Introduction

Session types describe the permitted sequences of method calls on a protocol-governed object [1, 2]. The state space of a session type, ordered by reachability, forms a lattice—a result central to the algebraic-reticulate theory [4].

The parallel composition operator $S_1 \parallel S_2$ introduces concurrent access: two independent execution paths on a shared object, whose state space is the product lattice $\mathcal{L}(S_1) \times \mathcal{L}(S_2)$. A natural and common pattern is to perform work in parallel and then continue sequentially:

$$(S_1 \parallel S_2) \,.\, S_3 \tag{1}$$

This note identifies two semantic problems with the grammar as currently formulated, proposes minimal resolutions, and proves that the resulting constructions preserve the lattice property.

2 Problem 1: Overloaded end

In all standard session type systems [1, 2, 3], **end** is the terminated session type:

> *After* **end**, *no further method calls are permitted.*

This is not merely a convention; it is a semantic invariant. The type system guarantees that no well-typed client can invoke a method on an object whose session type has reached **end**.

Now consider the parallel-then-continue pattern as it appears in practice:

$$(\mathit{read} \,.\, \mathbf{end} \parallel \mathit{write} \,.\, \mathbf{end}) \,.\, \mathit{close} \,.\, \mathbf{end} \tag{2}$$

The intended reading is: *read* and *write* execute concurrently; once both complete, *close* is called, and the session terminates. Seven of the thirty-four benchmark protocols in the reticulate tool [5] use this pattern, including the File Channel, Saga Orchestrator, and MQTT Client protocols.

The problem is that **end** inside the parallel branches and **end** at the end of the session carry *different semantic meanings*:

Occurrence	Intended meaning	Standard meaning
read . **end**	"read branch completed"	"session terminated"
write . **end**	"write branch completed"	"session terminated"
$\cdots$. *close* . **end**	"session terminated"	"session terminated"

The first two occurrences use **end** to mean *branch completion* (synchronisation), while the third uses it to mean *session termination* (finality). This is a semantic overload. Under the standard interpretation, *close* follows a terminated session, which is ill-formed.

3 Problem 2: The Sequencing Sugar

The grammar as previously defined includes the production $S_1 . S_2$ (sequencing) with a desugaring rule:

> When the left operand of (.) is a bare identifier m, desugar $m \,.\, S$ to $\&\{m : S\}$. Otherwise, produce a Sequence(S_1, S_2) node.

This means the dot operator (.) has two different behaviours depending on the syntactic form of its left operand—a context-sensitive rule embedded in what appears to be context-free syntax. This creates several problems:

(a) **Dual semantics of a single operator.** The expression $a \,.\, S$ produces a Branch, while $(S_1 \parallel S_2) \,.\, S$ produces a Sequence. The same syntactic operator yields different AST nodes.

(b) **Hidden relationship.** The equivalence $a \,.\, b \,.\, \mathbf{end} \equiv \&\{a : \&\{b : \mathbf{end}\}\}$ is not apparent from the syntax. A reader must know the desugaring rule to understand that method sequencing *is* branch with one arm.

(c) **Continuation scope is unclear.** In $\&\{a : \mathbf{end}, b : \mathbf{end}\} \,.\, c \,.\, \mathbf{end}$, does the continuation c follow the entire branch, or is it part of one arm? With explicit branch notation, this ambiguity does not arise.

3.1 Observation: Continuation is Exclusively for Parallel

We observe that the Sequence node is only semantically meaningful after parallel composition. For all other constructs, the continuation can—and should—be expressed inside the construct itself:

Construct	Continuation after?	Why not
$\&\{\ldots\} \,.\, S$	No	Push S inside each arm
$\oplus\{\ldots\} \,.\, S$	No	Push S inside each arm
$(\boldsymbol{\mu} X \,.\, S_1) \,.\, S_2$	No	Push S_2 inside the body
$(S_1 \parallel S_2) \,.\, S_3$	**Yes**	Shared synchronisation point

Parallel is the *only* construct where the continuation cannot be pushed inside, because the continuation is shared across both branches and must execute exactly once, after synchronisation. This makes continuation after parallel a fundamentally different operation from method sequencing.

4 Design Space

We considered three resolutions to the **end** overload, seeking the minimal change to the grammar.

4.1 Option A: Scoped end

Redefine **end** to mean "*this scope is complete*": inside $\|$, it signals branch completion; at the top level, it signals session termination. The grammar is unchanged (8 constructs).

Advantage. No new syntax.

Disadvantage. The same keyword carries two meanings, determined by context. This contradicts the standard interpretation in the literature and complicates both formal reasoning and user comprehension.

4.2 Option B: Explicit wait

Introduce a new keyword **wait** that means "*this parallel branch has completed; wait for siblings to finish before continuing.*" The grammar grows by one keyword (9 constructs).

Advantage. Each keyword has exactly one meaning. **end** is always terminal; **wait** is always synchronisation. Well-formedness rules are clear and local: **wait** outside $\|$ is an error; **end** followed by continuation is an error.

Disadvantage. One additional keyword.

4.3 Option C: Parallel with Explicit Continuation

Make parallel a ternary operator: $S_1 \| S_2 \; ; \; S_3$ means "execute S_1 and S_2 concurrently, then continue with S_3." This eliminates both **wait** and Sequence (8 constructs).

Advantage. No branch-completion marker needed.

Disadvantage. Parallel becomes a ternary construct, complicating the grammar. Terminal parallel ($S_1 \| S_2$ without continuation) and parallel-with-continuation ($S_1 \| S_2 \; ; \; S_3$) become separate productions, obscuring their relationship.

4.4 Decision

We adopt **Option B** together with the removal of the sequencing sugar. The combined effect is:

- The **end** overload is eliminated (Option B).
- The dot operator has a single, unambiguous meaning: continuation after parallel (sugar removal).
- Every construct does exactly one thing (Table 1).

Table 1: Each construct in the core grammar has exactly one purpose.

Syntax	AST node	Meaning
$\&\{m_i : S_i\}$	Branch	Method choice (one or many arms)
$\oplus\{l_i : S_i\}$	Select	Internal decision
$S_1 \parallel S_2$	Parallel	Concurrent execution
$S_1 \,.\, S_2$	Continuation	Resume after parallel sync
$\boldsymbol{\mu} X \,.\, S$	Rec	Recursive protocol
$(\, S \,)$	(grouping)	Precedence override
X	Var	Type variable reference
end	End	Session terminated (final)
wait	Wait	Branch completed (synchronise)

5 The Core Grammar

Definition 5.1 (Session type syntax — core grammar)**.** *The set of session types is generated by:*

$$
\begin{array}{rll}
S ::= & \&\,\{m_1 : S_1, \ldots, m_n : S_n\} & \textit{branch (external choice)} \\
\mid & \oplus\,\{l_1 : S_1, \ldots, l_n : S_n\} & \textit{selection (internal choice)} \\
\mid & S_1 \parallel S_2 & \textit{parallel composition} \\
\mid & S_1 \,.\, S_2 & \textit{continuation (after parallel)} \\
\mid & \boldsymbol{\mu} X \,.\, S & \textit{recursion} \\
\mid & (\, S \,) & \textit{grouping} \\
\mid & X & \textit{type variable} \\
\mid & \mathbf{end} & \textit{session terminated} \\
\mid & \mathbf{wait} & \textit{branch completed}
\end{array}
\tag{3}
$$

Definition 5.2 (Precedence)**.** *The continuation operator* (.) *binds tighter than parallel* ($\parallel$)*. Parentheses override precedence as usual.*

Remark 5.3 (No desugaring rules)**.** *The core grammar has no desugaring rules. Every syntactic form maps to exactly one AST node. Method sequencing—previously written as* $a \,.\, b \,.\, \mathbf{end}$ *and desugared to* $\&\{a : \&\{b : \mathbf{end}\}\}$*—is now written directly in branch notation:*

$$\&\{a : \&\{b : \mathbf{end}\}\}$$

This is more verbose but unambiguous. The sugar may be reintroduced as a notational convenience once the core theory is established (Section 9).

Remark 5.4 (Continuation is paired with parallel)**.** *The continuation operator* (.) *and the* **wait** *keyword are a* paired construct*:* **wait** *marks where parallel branches synchronise;* (.) *carries what happens after. A well-formedness constraint (Definition 6.1, condition v) enforces that the left operand of* (.) *is always a parallel composition.*

6 Well-Formedness

Definition 6.1 (Well-formedness rules)**.** *A session type* S *is* well-formed *if the following conditions hold:*

(i) ***Finality of* end*:*** **end** *does not appear as the left operand of continuation. That is,* $\mathbf{end} \,.\, S$ *is ill-formed for any* S*.*

(ii) ***Scope of* wait*:*** **wait** *appears only inside a parallel composition* $S_1 \parallel S_2$*. That is,* **wait** *at the top level or outside any* $\parallel$ *scope is ill-formed.*

(iii) ***Parallel branches with continuation:*** *If a parallel composition* $S_1 \| S_2$ *is the left operand of* (.)*, then every maximal path in* S_1 *and* S_2 *must terminate in* **wait***, not* **end***.*

(iv) ***Terminal parallel:*** *If a parallel composition* $S_1 \parallel S_2$ *is* not *the left operand of* (.)*, then every maximal path in* S_1 *and* S_2 *must terminate in* **end***, not* **wait***.*

(v) ***Continuation requires parallel:*** *The left operand of the continuation operator* (.) *must be a parallel composition (possibly parenthesised).*

Proposition 6.2 (Decidability)**.** *The well-formedness conditions of Definition 6.1 are decidable by a single pass over the abstract syntax tree.*

Proof. Each condition is a local, structural check. Condition (i) inspects continuation nodes. Condition (ii) maintains a Boolean flag tracking whether the current node is inside a Parallel scope. Conditions (iii) and (iv) inspect the leaves of parallel branches with knowledge of whether a continuation follows, which is determined at the parent node. Condition (v) inspects the left child of each continuation node. □

7 Lattice Preservation

The key theoretical question is whether the parallel-then-continue pattern (1) preserves the lattice property of the state space.

Definition 7.1 (State-space construction for continuation)**.** *Let* $\mathcal{L}_1 = \mathcal{L}(S_1 \parallel S_2)$ *be the product lattice of the parallel block and* $\mathcal{L}_2 = \mathcal{L}(S_3)$ *the state space of the continuation. The state space of* $(S_1 \parallel S_2)\ .\ S_3$ *is the disjoint union* $\mathcal{L}_1 \sqcup \mathcal{L}_2$ *with a transition from* $\bot_{\mathcal{L}_1}$ *(the synchronisation point where all branches have reached* **wait***) to* $\top_{\mathcal{L}_2}$ *(the initial state of the continuation).*

Theorem 7.2 (Ordinal sum preserves lattice)**.** *If* $\mathcal{L}_1$ *and* $\mathcal{L}_2$ *are finite lattices, then the state space of* $\mathcal{L}_1\ .\ \mathcal{L}_2$ *as constructed in Definition 7.1 is a finite lattice.*

Proof. The reachability order on $\mathcal{L}_1\ .\ \mathcal{L}_2$ satisfies:

- For $x, y \in \mathcal{L}_1$: $x \geq y$ iff $x \geq_{\mathcal{L}_1} y$.
- For $x, y \in \mathcal{L}_2$: $x \geq y$ iff $x \geq_{\mathcal{L}_2} y$.
- For $x \in \mathcal{L}_1$, $y \in \mathcal{L}_2$: $x \geq y$ always (since $x \geq_{\mathcal{L}_1} \bot_{\mathcal{L}_1} \to \top_{\mathcal{L}_2} \geq_{\mathcal{L}_2} y$).
- For $x \in \mathcal{L}_2$, $y \in \mathcal{L}_1$: $x \geq y$ never (no backward edge).

This is the *ordinal sum* $\mathcal{L}_1 \oplus_{\text{ord}} \mathcal{L}_2$, a standard construction in lattice theory [6, 7]. We verify that all pairwise meets and joins exist.

Case 1: $x, y \in \mathcal{L}_1$. Since $\mathcal{L}_1$ is a lattice, $x \wedge_{\mathcal{L}_1} y$ and $x \vee_{\mathcal{L}_1} y$ exist. These are also the meet and join in the combined order, because no element of $\mathcal{L}_2$ can be an upper bound of any element of $\mathcal{L}_1$.

Case 2: $x, y \in \mathcal{L}_2$. Symmetric argument: $\mathcal{L}_2$ is a lattice, and no element of $\mathcal{L}_1$ can be a lower bound of any element of $\mathcal{L}_2$ other than via the existing order in $\mathcal{L}_2$.

Case 3: $x \in \mathcal{L}_1$, $y \in \mathcal{L}_2$. Since $x \geq y$:

- $\text{meet}(x, y) = y$, because y is a lower bound of $\{x, y\}$ and any $z \leq x$ with $z \leq y$ must satisfy $z \in \mathcal{L}_2$ (elements of $\mathcal{L}_1$ other than y itself are all $\geq y$, hence not $\leq y$), so $z \leq_{\mathcal{L}_2} y$, giving y as greatest.
- $\text{join}(x, y) = x$, by symmetric reasoning: any $z \geq y$ with $z \geq x$ must satisfy $z \in \mathcal{L}_1$ (elements of $\mathcal{L}_2$ cannot reach x), so $z \geq_{\mathcal{L}_1} x$, giving x as least.

The top of the combined lattice is $\top_{\mathcal{L}_1}$ and the bottom is $\bot_{\mathcal{L}_2}$. All pairwise meets and joins exist; hence the combined poset is a lattice. □

Corollary 7.3. *For any well-formed session types S_1, S_2, S_3: if $\mathcal{L}(S_1 \parallel S_2)$ is a lattice and $\mathcal{L}(S_3)$ is a lattice, then $\mathcal{L}((S_1 \parallel S_2) \,.\, S_3)$ is a lattice.*

Proof. By Theorem 7.2, since the product lattice $\mathcal{L}(S_1) \times \mathcal{L}(S_2)$ is a lattice (products of lattices are lattices) and $\mathcal{L}(S_3)$ is a lattice by assumption. □

Corollary 7.4. *The ordinal sum is associative: continuation of any finite chain of lattices is a lattice. Thus patterns such as $(S_1 \parallel S_2) \,.\, (S_3 \parallel S_4) \,.\, S_5$ are lattices.*

8 Examples

We illustrate the core grammar on representative benchmarks. All examples use explicit branch notation (no sequencing sugar).

Example 8.1 (File Channel)**.** ***Before*** *(sugar, overloaded* **end***):*

$$(\mathit{read} \,.\, \mathbf{end} \parallel \mathit{write} \,.\, \mathbf{end}) \,.\, \mathit{close} \,.\, \mathbf{end}$$

After *(core grammar):*

$$(\&\{\mathit{read} : \mathbf{wait}\} \parallel \&\{\mathit{write} : \mathbf{wait}\}) \,.\, \&\{\mathit{close} : \mathbf{end}\}$$

State space: 5 states, 5 transitions. Lattice: ✓.

Example 8.2 (Saga Orchestrator)**.** ***Core grammar:***

$$(\&\{\mathit{step_1} : \oplus\{\mathit{OK} : \mathbf{wait},\ \mathit{FAIL} : \&\{\mathit{comp_1} : \mathbf{wait}\}\}\} \parallel$$
$$\&\{\mathit{step_2} : \oplus\{\mathit{OK} : \mathbf{wait},\ \mathit{FAIL} : \&\{\mathit{comp_2} : \mathbf{wait}\}\}\}) \,.\, \oplus\{\mathit{ALL_OK} : \&\{\mathit{commit} : \mathbf{end}\},\ \mathit{PARTIAL} : \&\{\mathit{rollbac}$$

Example 8.3 (Terminal parallel (no continuation))**.** *When parallel is the final construct, branches use* **end** *as usual. No* **wait** *is needed or permitted:*

$$\&\{\mathit{read} : \mathbf{end}\} \parallel \&\{\mathit{write} : \mathbf{end}\}$$

Example 8.4 (Simple linear protocol (no parallel))**.** *Without the sequencing sugar, a linear protocol uses nested branches:*

$$\&\{\mathit{open} : \&\{\mathit{read} : \&\{\mathit{close} : \mathbf{end}\}\}\}$$

Previously written as $\mathit{open} \,.\, \mathit{read} \,.\, \mathit{close} \,.\, \mathbf{end}$*. The explicit form is more verbose but unambiguous.*

9 Notational Sugar (Deferred)

The core grammar is deliberately minimal to simplify formal reasoning. Once the theory is established, the sequencing sugar may be reintroduced as a notational convenience:

Notation. We write $m \,.\, S$ as shorthand for $\&\{m : S\}$, where m is a method name.

This is a purely syntactic abbreviation that does not affect the AST, the state-space construction, or any formal results. The sugar satisfies a simple invariant: it is only a shorthand for single-arm branches, and it never produces a Continuation node.

With the sugar restored, Example 8.1 may be written:

$$(\mathit{read} \,.\, \mathbf{wait} \parallel \mathit{write} \,.\, \mathbf{wait}) \,.\, \mathit{close} \,.\, \mathbf{end}$$

Note that the sugar applies inside branches (mapping $m \,.\, S$ to $\&\{m : S\}$) but the top-level . after the parallel block remains the continuation operator. The two uses of the dot are now distinguished by context: identifier on the left means sugar; parallel on the left means continuation.

10 Implementation Plan

We outline the changes required to implement the core grammar in the reticulate tool (Python) and BICA Reborn (Java).

10.1 Phase 1: AST and Parser

1. **New AST node**: Add a `Wait` node (frozen dataclass / sealed record), analogous to `End`.
2. **Keyword registration**: Register `wait` as a keyword in the tokenizer, alongside `end` and `rec`.
3. **Remove sequencing sugar**: The parser rule for $ident.S \to \text{Branch}((ident, S))$ is removed. Bare identifiers followed by a dot are now parse errors; users must write $\&\{m : S\}$ explicitly.
4. **Restrict continuation**: The dot operator (.) only produces a Continuation node (renamed from Sequence). A well-formedness check (or parser-level restriction) ensures the left operand is a Parallel expression.
5. **Pretty-printer**: Update to emit **wait** and use explicit branch notation.

10.2 Phase 2: State-Space Construction

1. **Wait semantics**: In `build_statespace`, a **wait** node produces a terminal state (no outgoing transitions), just like **end**. The distinction between **wait** and **end** is enforced at the well-formedness level, not the state-space level.
2. **Continuation construction**: The existing Sequence handling (chain $\bot_{\mathcal{L}_1} \to \top_{\mathcal{L}_2}$) is retained, now named "continuation" for clarity.

10.3 Phase 3: Well-Formedness Checker

1. Implement the five conditions of Definition 6.1 as a single-pass AST visitor.
2. Integrate into the CLI pipeline: parse $\to$ well-formedness $\to$ state space $\to$ lattice check.
3. Report clear error messages for each violation (e.g., "`wait` outside parallel scope at position n").

10.4 Phase 4: Benchmark Migration

1. Rewrite the 7 benchmarks that use the continuation pattern: replace **end** inside parallel branches with **wait**.
2. Rewrite all 34 benchmarks to use explicit branch notation (remove sequencing sugar).
3. Verify: all benchmarks parse, pass well-formedness, build correct state spaces, and remain lattices.

10.5 Phase 5: Documentation and Tests

1. Update the parser module docstring and grammar documentation.
2. Update the man page (`reticulate(1)`).
3. Update the user manual.
4. Add tests for: `Wait` parsing, well-formedness violations (all five conditions), continuation state-space construction, lattice preservation across continuation.

11 Summary

	Before	**After (core grammar)**
Grammar constructs	8	9
Keywords	**end**, **rec**	**end**, **wait**, **rec**
Desugaring rules	1 ($m \,.\, S \to \&\{m : S\}$)	0
Semantic overload of **end**	yes	no
Dot operator meanings	2 (sugar + continuation)	1 (continuation only)
Synchronisation marker	implicit (scoped **end**)	explicit (**wait**)
Continuation restricted to	any compound expr	parallel only
Well-formedness rules	context-dependent	local, structural
Lattice preservation	✓ (ordinal sum)	✓ (ordinal sum)

The combined introduction of **wait** and removal of the sequencing sugar yields a core grammar in which every construct has exactly one meaning, no desugaring rules are needed, and the continuation operator is exclusively paired with parallel composition. The lattice property is preserved by the ordinal-sum construction (Theorem 7.2), ensuring that the algebraic-reticulate theory extends smoothly to the refined grammar.

The sequencing sugar, when it is reintroduced as a notational convenience, will be explicitly marked as such—a syntactic abbreviation with no impact on the formal foundations.

References

[1] K. Honda, V. T. Vasconcelos, and M. Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP*, volume 1381 of LNCS, pages 122–138. Springer, 1998.

[2] S. J. Gay and M. J. Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005.

[3] V. T. Vasconcelos. Fundamentals of session types. *Information and Computation*, 217:52–70, 2012.

[4] A. Z. Caldeira. Session type state spaces form lattices. Manuscript, 2026.

[5] A. Z. Caldeira. Reticulate: A lattice checker for session types. Manuscript, 2026.

[6] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002.

[7] G. Grätzer. *Lattice Theory: Foundation*. Birkhäuser, 2011.